

Compositional (Semantic) Models and Unsupervised Graph Structure Learning

Lecture 12, part 2

Notes for Probabilistic Models of the Visual Cortex

Alan Yuille

Johns Hopkins University

8 October, 2026

What this lecture covers

Compositional models

AND/OR graphs

Learning from interest points

Learning a hierarchy

Part sharing for many objects

Summary

Colours used in the figures

In the graph diagrams drawn for these notes, colour has one meaning throughout. In the other drawings and in the charts, colours only tell elements apart, and the captions say what they mark. The pictures taken from the papers keep their own colours, which the captions explain. A caption that says “CMUGL p. N ” took its picture from page N of the earlier course slides on compositional models and unsupervised graph learning (Yuille, CMUGL); the text calls them the CMUGL slides.

- ▶ **Blue** marks the nodes of a model: AND nodes, parts.
- ▶ **Teal** marks shared parts and the lateral relations between siblings.
- ▶ **Orange** marks what is new or unknown: new triplets, unknown states, candidate correspondences, OR nodes, and the bottom-up direction.
- ▶ **Purple** marks the top-down direction.
- ▶ **Red** marks rejected proposals and thresholds.

Compositional models make parts and relations explicit

The first part of this lecture used CompNets: generative models of the features of a deep network, in which an object is a spatial template of parts, one template per viewpoint. The parts are the von Mises-Fisher (vMF) kernels, and their arrangement is a map of mixture weights. There is one level of parts, and the relations between them are implicit in the template.

This part goes back to an older line of work (about 2006 to 2011, by Long (Leo) Zhu, Yuanhao Chen and Alan Yuille) in which the composition is explicit and deep. An object is built from parts, each part from subparts, and so on down to edges in the image; the parts are nodes of a graph, and their spatial relations are edges with their own probability terms. The graph itself can be learnt from images.

This work went out of fashion when deep networks took over. The ideas are presented here because compositionality is key to intelligence, and they are likely to return: explicit parts give interpretability, the ability to perform many tasks with one model, and robustness when the data change.

Objects and semantic parts

Objects can be represented by parts and their spatial relations. The idea is old: the pictorial structures of Fischler and Elschlager (1973) describe a face by a few parts (eyes, nose, mouth, hair, the edges of the face) held in place by springs. Each part has a model of its appearance, and each spring has a preferred length; the springs can stretch, so the same model fits faces whose parts are a little further apart or a little closer together.

Hierarchical semantic compositional models go further: objects are made of parts, parts of subparts, and so on, in a hierarchy of several levels. A face is made of eyes, a nose and a mouth; an eye is made of an iris, a pupil and eyelids.

Semantic means that the parts are interpretable: an eye or a nose, not an arbitrary patch of the object such as its top-right quarter. Semantic composition means that an object, or a part, is composed of interpretable parts or subparts, rather than combined by an arbitrary mathematical operation.

Objects and semantic parts

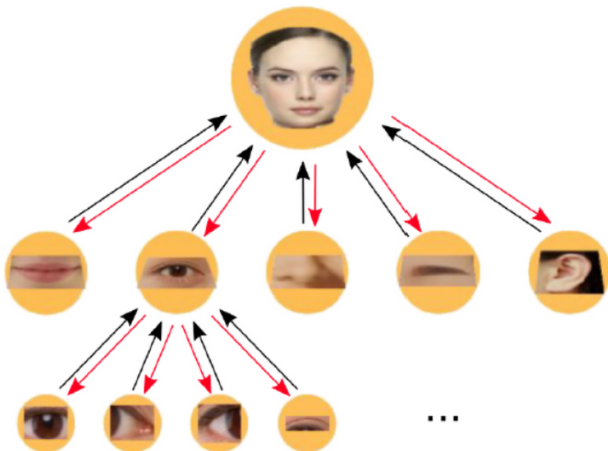


Figure 1: A hierarchical semantic compositional model of a face (from CMUGL p. 3): the face is composed of parts (lips, an eye, the nose, an eyebrow, an ear), and the eye of subparts. Black arrows run bottom-up, red arrows top-down.

Pictorial structures hold the parts together with springs

A pictorial structure is the simplest compositional model, with one level of parts. Matching a pictorial structure to an image is an optimisation. Each part i is placed at a position x_i ; the cost of a placement is the sum of two kinds of terms:

$$E(x_1, \dots, x_n) = \sum_i a_i(x_i) + \sum_{(i,j)} s_{ij}(x_i, x_j).$$

The first sum runs over the parts, the second over the pairs of parts joined by a spring. The appearance cost $a_i(x_i)$ is small where the image looks like part i ; the spring cost s_{ij} grows as the offset $x_j - x_i$ moves away from its preferred value. The best match minimises E . Lecture 6 met the same form for the labels of the pixels of an image: unary terms from the data and pairwise terms from the context. Written as $P \propto e^{-E}$, it is a probability model on the graph whose nodes are the parts and whose edges are the springs.

Pictorial structures hold the parts together with springs

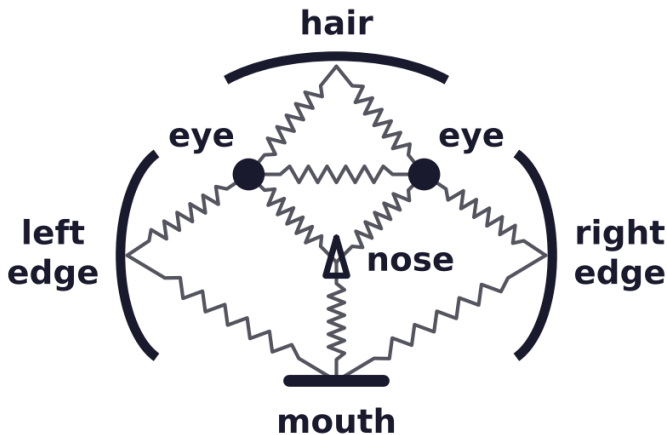


Figure 2: A face as a pictorial structure, after Fischler and Elschlager (1973), redrawn: the parts (hair, eyes, nose, mouth, left and right edges of the face) are connected by springs.

A hierarchical compositional model is a graph of parts

Formally, a hierarchical compositional model (HCM) is represented by a hierarchical graph. The figure gives an example for a horse: the object node at the top; parts below it (head, body, legs); subparts below the parts; and, at the bottom, the image, where the subparts are matched to edge features. Solid lines join each node to its children; dashed teal lines join siblings, the children of one parent.

Such models have several properties.

- ▶ They are explainable: if an object is detected, you can explain which parts and subparts are present, and where.
- ▶ They perform multiple tasks with one model: detect and localize objects, detect and localize parts and subparts, and detect and localize the object boundaries.
- ▶ Inference is bottom-up and top-down, and requires sideways (lateral) reasoning between siblings.
- ▶ The parts, the subparts and the relationships between them need to be learnt.

A hierarchical compositional model is a graph of parts

Bottom-up inference proposes: edge features in the image suggest subparts, groups of subparts suggest parts, and groups of parts suggest the object. Top-down inference validates: once a horse is hypothesised, it predicts where its legs should be, and a leg that was missed bottom-up (because it is faint or partly hidden) can be found. Lateral reasoning checks that siblings are in a consistent arrangement: the head at the front end of the body and the legs below it.

This is a different division of labour from a feed-forward deep network, which computes its features bottom-up only. Explicit parts give advantages for out-of-distribution learning and domain adaptation: a part that is occluded or changed can be switched off or relearned while the rest of the model is kept, as with the CompNets of the first part. The structure also has a justification in cognitive science and neuroscience, where vision is organised in levels.

A hierarchical compositional model is a graph of parts

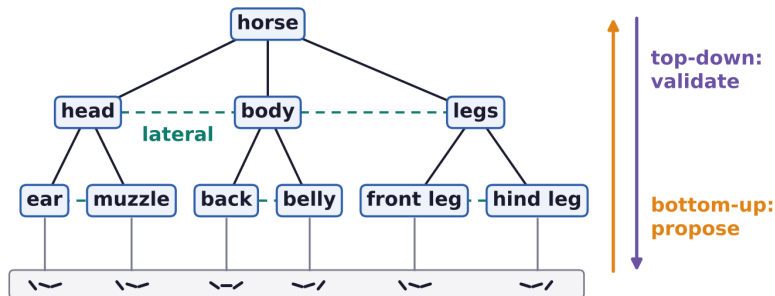


Figure 3: A hierarchical compositional model of a horse. Rows, from the top: the object, its parts, its subparts, and the image, where the subparts meet edge features (short strokes; grey lines join them to the subparts). Solid lines: parent and children; dashed teal lines: lateral relations between siblings. Orange arrow: bottom-up inference proposes; purple arrow: top-down inference validates.

Hierarchies follow low-, middle- and high-level vision

Why hierarchies? Vision has a low-, middle- and high-level nature.

Low-level vision is ambiguous: a small patch of an image can be an edge, a shadow, a texture or noise, and it cannot be decided from the patch alone. High-level vision exploits context and is unambiguous: in the whole image there is no doubt that the patch is the lower rim of a cat's eye.

A hierarchy lets the levels help each other. Low-level features are combined into middle-level structures (an eye), and these into objects (a cat); the high-level interpretation then resolves the ambiguities that remained at the lower levels, top-down.

Hierarchies mimic the structure of the human and primate ventral visual system, the pathway from the primary visual cortex (V1) through V2 and V4 to the inferotemporal cortex, along which the receptive fields grow and the neurons respond to increasingly complex patterns. The CMUGL slides ask: are hierarchies an optimal design for representing, learning and retrieving image patterns?

Hierarchies follow low-, middle- and high-level vision

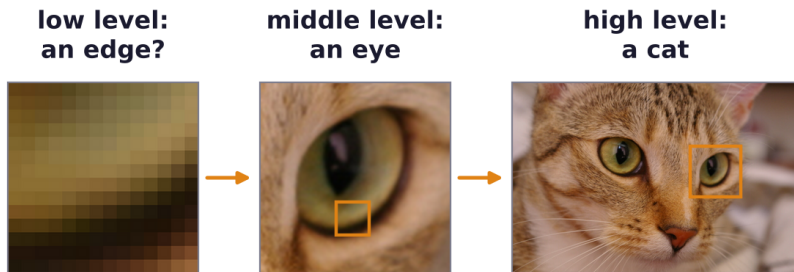


Figure 4: Three levels on one photograph (scikit-image “chelsea”): a 14×14 pixel patch, enlarged (low level: an edge?), the 80×80 eye region (middle level: an eye) and the whole image (high level: a cat). The orange boxes show where each smaller view comes from.

A probabilistic model is defined by four elements

A probabilistic model on a graph is defined by four elements (Zhu, Chen and Yuille, 2011, a review of these models).

- (i) The graph structure: nodes $\mathcal{V}$ and edges $\mathcal{E}$. This is part of the representation.
- (ii) State variables $\mathbf{W}$: a state w_μ at every node μ , for example the position, scale and orientation of the part. These are the representation of the image content.
- (iii) Potentials ϕ : functions of the states of the nodes that an edge joins, and of the image. They define the probability.
- (iv) Parameters (weights) λ : how much each potential counts.

The state variables are defined at the graph nodes. The potentials and parameters are defined over the graph edges, and they also relate the model to the image $\mathbf{I}$. The figure marks the three kinds of potential on the horse hierarchy: vertical ones between a parent and its children, horizontal ones between siblings, and data potentials between a node and the image.

A probabilistic model is defined by four elements

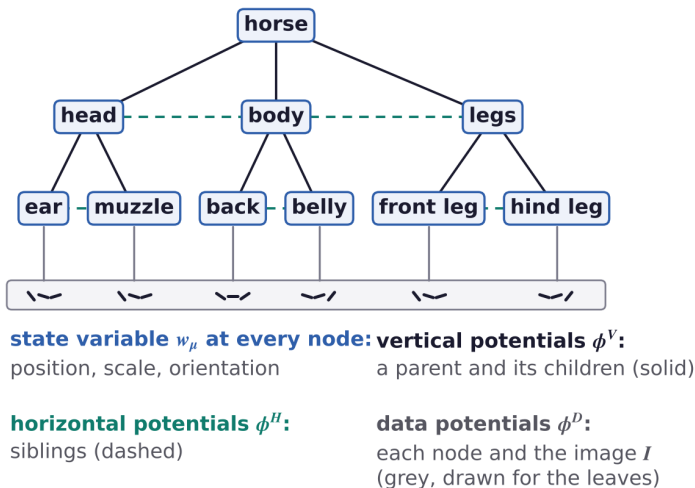


Figure 5: The four elements on the horse hierarchy. Each node carries a state w_μ ; solid lines are vertical potentials, dashed teal lines horizontal potentials, grey lines data potentials (drawn for the leaves only; every node has one).

The mathematics: an exponential model on the graph

The mathematical formulation uses exponential models. The graph is $(\mathcal{V}, \mathcal{E})$; $\mathcal{V}^l$ are the nodes at level l ; a node μ at level l has children $ch(\mu) \subset \mathcal{V}^{l-1}$ and siblings $sib(\mu) \subset \mathcal{V}^l$. The states of all nodes are $\mathbf{W} = \{w_\mu\}$, and

$$P(\mathbf{W} | \mathbf{I}) = \frac{1}{Z} \exp \left\{ \sum_{\mu \in \mathcal{V}} \lambda_\mu^D \cdot \phi^D(w_\mu, \mathbf{I}) + \sum_{\mu \in \mathcal{V}} \lambda_\mu^V \cdot \phi^V(w_\mu, w_{ch(\mu)}) + \sum_{\mu \in \mathcal{V}} \lambda_\mu^H \cdot \phi^H(w_\mu, w_{sib(\mu)}) \right\}.$$

- ▶ Data potentials $\phi^D(w_\mu, \mathbf{I})$ relate node μ to the image: is there evidence for the part at the state w_μ ?
- ▶ Vertical potentials $\phi^V(w_\mu, w_{ch(\mu)})$ relate a part to its subparts: are the children where the parent expects them?
- ▶ Horizontal potentials $\phi^H(w_\mu, w_{sib(\mu)})$ relate a part to its siblings.

Each potential can be a vector of features, and the weights λ are vectors of the same length; the dot says how much each feature counts.

The mathematics: an exponential model on the graph

The normaliser $Z = Z[\lambda, \mathbf{I}]$ sums the exponential over all states $\mathbf{W}$, so that the probabilities sum to one. It depends on the image and on all the weights. This is a Gibbs distribution,

$$P(\mathbf{W} | \mathbf{I}) = \frac{1}{Z} e^{-E(\mathbf{W}, \mathbf{I})}, \quad E(\mathbf{W}, \mathbf{I}) = - \sum_{\mu} [\lambda_{\mu}^D \cdot \phi^D + \lambda_{\mu}^V \cdot \phi^V + \lambda_{\mu}^H \cdot \phi^H],$$

with an energy E that is a sum of local terms on the graph. Both this lecture and the Markov random fields of Lecture 6 write $P \propto e^{-E}$. In Lecture 6 the terms of E were costs, small for a good fit. Here E is minus a weighted sum of potentials, so a large $\lambda \cdot \phi$ means a good fit and a low energy.

An example of the terms: the vertical potential of the horse's body can be large when its children (back, belly) lie at their expected offsets from the body, scaled and rotated with it, like the springs of a pictorial structure; the data potential of a leaf can be large when an edge of the right orientation lies at its position.

Three tasks: inference, learning and structure induction

Three tasks arise.

- (I) Inference: estimate the state $\mathbf{W}$ from the input $\mathbf{I}$, assuming that the graph structure, the potentials and the parameters are known.
Intuitively: propagate hypotheses up the hierarchy and validate them top-down.
- (II) Learning the parameters and potentials, assuming that the graph structure is known. This is straightforward if inference can be done: the parameters can be fitted, for example by maximum likelihood or by a max-margin criterion (weights chosen so that the correct parse of each training image beats every other parse by a margin), using inference to find the best states of the training images.
- (III) Structure induction: learn the graph structure itself, the nodes and the edges. This is the hardest task; learning from interest points and learning a hierarchy are two ways to do it.

Three tasks: inference, learning and structure induction

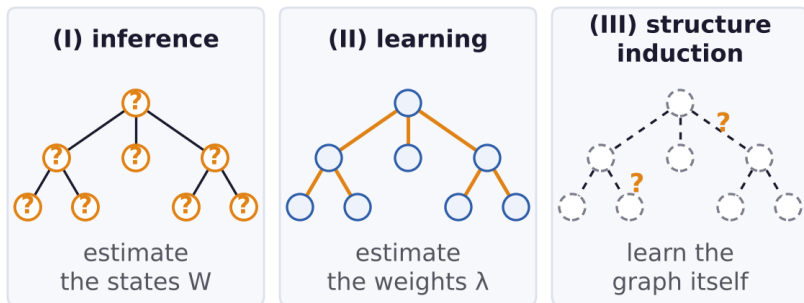


Figure 6: The three tasks. (I) Inference: the graph and the weights are known, the states (orange question marks) are unknown. (II) Learning: the graph is known, the weights on the edges (orange) are unknown. (III) Structure induction: the graph itself (dashed) is unknown.

Dynamic programming parses a tree exactly

Inference (I) can be exact when each node has one parent, so that the parent-child edges form a tree. The most probable state is then computed by dynamic programming (Lecture 9). Going bottom-up, each node μ computes, for each of its own states w_μ , the best score $h_\mu(w_\mu)$ of its whole subtree:

$$h_\mu(w_\mu) = \lambda_\mu^D \cdot \phi^D(w_\mu, \mathbf{I}) + \max_{w_{ch(\mu)}} \left[\lambda_\mu^V \cdot \phi^V(w_\mu, w_{ch(\mu)}) \right. \\ \left. + \sum_{\nu \in ch(\mu)} (\lambda_\nu^H \cdot \phi^H(w_\nu, w_{sib(\nu)}) + h_\nu(w_\nu)) \right].$$

The maximum is over the joint states of all the children of μ , because the vertical and horizontal terms couple them. At the root r , the best state maximises h_r ; going top-down, the children of each node take the joint state that achieved the maximum for their parent's chosen state. This is “propagate hypotheses up, validate them top-down” made exact.

Dynamic programming parses a tree exactly

The cost of the recursion grows with the number of joint states of a family: with c children of k states each, the maximum in h_μ compares k^c joint states for each state of the parent. When the vertical potential splits into one term per child and there are no horizontal terms, the maximum splits into one maximum per child. With many states per node, the papers prune: they keep only the proposals whose score exceeds a threshold, and the recursion becomes approximate. Exact dynamic programming is the special case for trees whose nodes have few states.

Along a chain, where each node has one child, the recursion is the dynamic programming of Lecture 9; a tree replaces the single child by a family of children.

Compositionality: objects are built from shared parts

The key idea is compositionality: objects and images are constructed by compositions of parts, using two kinds of node.

- ▶ An AND node is made of all of its children: an object is a head AND a body AND legs.
- ▶ An OR node is one of its children: a leg is seen from the side OR from the front; the image shows object A OR object B.

The probability models are built in the same way: elementary models of small parts are combined by composition into models of larger parts, and these into models of objects.

The figure (Zhu, Chen, Torralba, Freeman and Yuille, 2010; cited as Zhu et al., 2010) shows why sharing matters. An object A is represented by a recursive compositional model (RCM) made of parts a and b , which are themselves more elementary RCMs; an object B is made of parts b and p . Represented separately, part b appears twice. Represented together, under an OR node that chooses between A and B, part b is shared: it is stored once and its evidence is computed once. Sharing parts makes inference and learning efficient.

Compositionality: objects are built from shared parts

Image grammars (S.-C. Zhu and Mumford, 2007) borrow the stochastic grammars of natural language processing. A sentence is composed of phrases, a phrase of words; a grammar says which compositions are allowed (the AND rules) and which alternatives exist (the OR rules), with probabilities on the alternatives. A parse of a sentence is a tree that applies the rules. An image grammar does the same for objects, parts and edges, with the additional complication that the parts must also fit together in space.

Grammars give two advantages that the CMUGL slides stress. Explicit representations make it possible to perform multiple tasks with one model: the parse of an image says where the object is, where its parts are, and where its boundary runs. And sharing makes both inference and learning efficient.

Compositionality: objects are built from shared parts

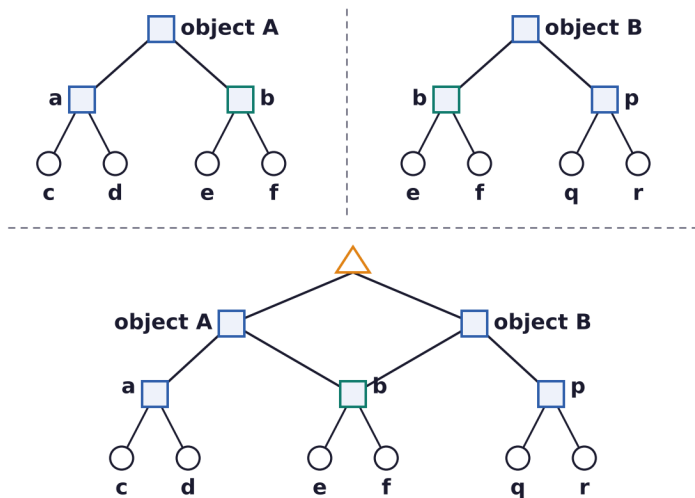


Figure 7: Part sharing (after Zhu et al., 2010, Fig. 2). Squares are AND nodes, circles elementary models, the triangle an OR node. First row: objects A and B as separate models, both with part *b* (teal). Second row: A and B together under an OR node, with part *b* shared.

Compositionality lets a system transfer, reason and scale up

Why compositionality? The CMUGL slides list four abilities.

1. Transfer between contexts, and generalize or extrapolate: a model of a cow, made of a body, legs, a head and horns, can be turned into a model of a yak by changing a few parts.
2. Reason about the system, intervene and do diagnostics: if a detection fails, the parts say why (the head was not found), and a part can be changed without retraining the rest.
3. Answer many different questions based on the same underlying knowledge structure: where is the object, where is its head, is it facing left, which part is hidden.
4. Scale up to multiple objects by part sharing: a new object reuses the parts that are already learnt.

Yuille and Liu (2021) describe the principle as “An embodiment of faith that the world is knowable, that one can tease things apart, comprehend them, and mentally recompose them at will.”

Compositionality lets a system transfer, reason and scale up

The second quotation, “The world is compositional or God exists”, is Stuart Geman’s, as quoted by Yuille and Liu (2021). Their reading: if the world were not compositional, there would be no way to learn it from limited data, and it would seem necessary for God to handwire human intelligence.

The argument behind both quotations is combinatorial. The number of possible images, and of possible arrangements of objects, is astronomically large; no system can learn each separately. A compositional system learns a moderate number of parts and the rules for combining them, and the combinations then cover the exponentially many cases, as the occlusion patterns of the first part of the lecture were covered by part-wise outlier decisions.

A horse model with AND nodes only

The horse model (Zhu, Chen and Yuille, 2010) is a hierarchy of AND nodes. Nodes of the graph represent parts of the object; the lower-level parts are edges and groupings of edges on the boundary of the horse, and the higher levels group them into larger pieces of the outline.

Parts can move and deform: each node has a state $w = (\text{position, scale, orientation})$. The vertical potentials prefer the children of a node to lie at their usual positions relative to the parent, transformed by the parent's scale and orientation; the data potentials of the leaves prefer edges of the right orientation at their positions.

A model with AND nodes only has a single topology: every node is always present, in the same arrangement. So one model describes one pose and one viewpoint. A horse that turns its head, rears or is seen from the front needs another model, and this is what OR nodes provide.

A horse model with AND nodes only

$w = (\text{position, scale, orientation})$ at every node

leaves: edges and edge-groupings on the boundary

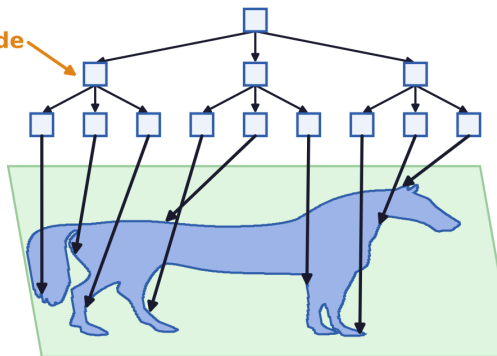


Figure 8: The horse model with AND nodes only (after CMUGL p. 13, redrawn with the scikit-image horse silhouette). Squares are nodes; the arrows of the leaves point to edges and edge groupings on the boundary of the horse. The image lies in the green plane.

AND/OR graphs switch between parts for different poses

AND/OR graphs introduce OR nodes and switch variables. Each OR node ν has a switch variable t_ν that selects one of its children, with a prior probability for each choice. Setting the switch variables alters the graph topology: it keeps the selected children and removes the others. What remains is an AND graph, a parse tree, with different parts for different viewpoints and poses.

An AND/OR graph is therefore a mixture of models with shared parts. The probability of a state is the probability of the parse tree that the switches $t = \{t_\nu\}$ select, times the prior $P(t)$ of the switch settings. The number of parse trees grows exponentially with the number of OR nodes: K OR nodes with n choices each give up to n^K trees. Yet the graph stores each part once, and inference can share the computation of the parts between all the trees in which they occur.

In the figure, rounded boxes and circles are OR nodes, square-cornered boxes AND nodes; curved lines join OR nodes of one row (lateral relations), and dots stand for more nodes. The small pictures show the part that each node represents, as coloured points on the boundary of the horse.

AND/OR graphs switch between parts for different poses

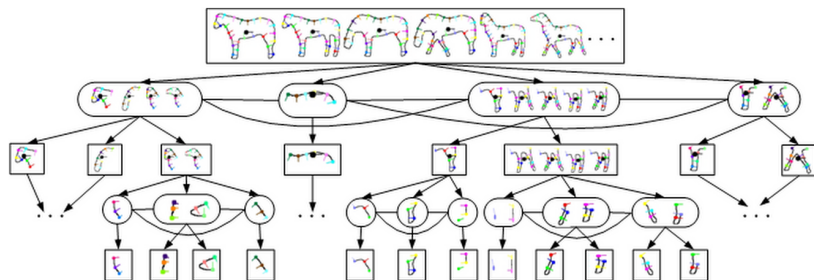


Figure 9: An AND/OR graph for horses (from CMUGL p. 14; Zhu, Chen, Lin and Yuille, 2011). Rounded boxes and circles: OR nodes; square-cornered boxes: AND nodes. The top row shows horses in different poses; each lower node shows its part as coloured points on the horse's boundary.

AND/OR graphs parse baseball players in 98 poses

AND/OR graphs enable recursive compositional models (RCMs) to deal with objects with many poses and viewpoints. Zhu, Chen, Lu, Lin and Yuille (CVPR 2008) built one for baseball players: its parse trees describe 98 poses. The arms are not modelled. As in the horse graph, rounded boxes and circles are OR nodes and square-cornered boxes AND nodes; each node shows examples of the body parts it covers, with one colour of dot per body part.

Inference and learning use bottom-up and top-down processing. The parameters are learnt by a max-margin criterion: the weights are chosen so that, on each training image, the correct parse scores higher than every other parse by a margin, as in a support vector machine. The model was trained on 156 images collected from the internet, and the paper reports its results on a standard set of 48 baseball images (2008, and in the journal version, Zhu, Chen, Lin and Yuille, 2011).

AND/OR graphs parse baseball players in 98 poses

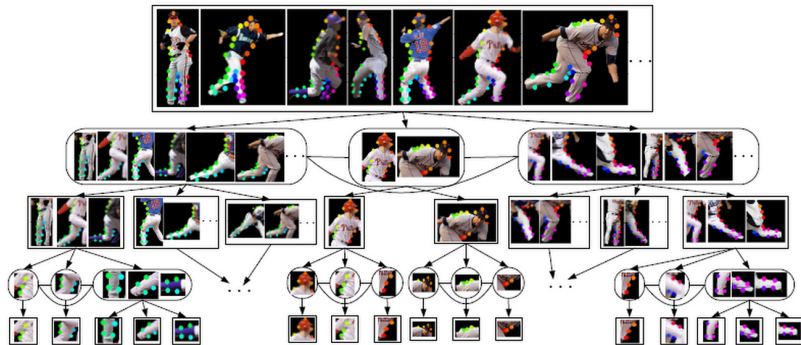


Figure 10: The AND/OR graph for baseball players (Zhu et al., CVPR 2008, from CMUGL p. 15). Rounded boxes and circles: OR nodes; square-cornered boxes: AND nodes; each node shows examples of its body parts.

The parses mark the positions of the body parts

The parse of a test image gives the state of every node of the selected parse tree, and so the position of every modelled body part. The figure shows results on test images: the coloured dots are the positions of body parts found by the model (head, torso, legs; the arms are not modelled). The pitchers and batters appear in very different poses, which a single AND graph could not describe.

The parses mark the positions of the body parts



Figure 11: Parsing results on baseball images (Zhu et al., CVPR 2008, from CMUGL p. 16). Coloured dots: positions of body parts.

HCMs are hard to learn but worth pursuing

Hierarchical compositional models (HCMs) bring challenges as well as promise.

- ▶ HCMs are significantly more complex than deep networks, in terms of inference and learning algorithms. A deep network computes one feed-forward pass; an HCM searches over the states of many nodes, bottom-up and top-down.
- ▶ Structure learning is particularly difficult: the graph is not given, and the number of possible graphs is enormous.
- ▶ Their advantages (interpretability, multi-tasking) and their potential (the ability to deal with domain transfer and with sophisticated attacks) are so strong that this class of models should be pursued.

CompNets, the first part of this lecture, combine compositional models with deep network features: the deep network supplies features that are invariant to low-level details, and the compositional model supplies explicit parts, an outlier model and spatial structure.

Learn models of simple features: interest points, edgelets

Unsupervised structure learning is an extremely challenging task: the graph of the model must be learnt from images, without being told what the parts are. The intuition for structure learning is clustering: parts are patterns that recur in the training images, in recurring spatial arrangements.

Learning generative models of entire images was too hard at the time of this work (2008), and structure induction is very hard. To simplify, the models are learnt for objects (not images), on simple features:

- ▶ interest points (IPs): points at a well-defined position, with an attribute of scale, that can be found again when the image is changed by photometric and geometric transformations;
- ▶ edgelets: short pieces of edge, each with a position and an orientation.

IPs are sparse, far fewer than the pixels, and they capture the important (interesting) parts of the object: corners, blobs, the centres of eyes and wheels.

Learn models of simple features: interest points, edgelets

An interest operator decides where the interest points are. The Kadir-Brady operator (Kadir and Brady, 2001) selects regions whose distribution of intensities has high entropy, at the scale where that entropy peaks: salient regions. The SIFT operator (scale-invariant feature transform; Lowe, 2004) selects the extrema of a difference of Gaussian-blurred images, in position and in scale.

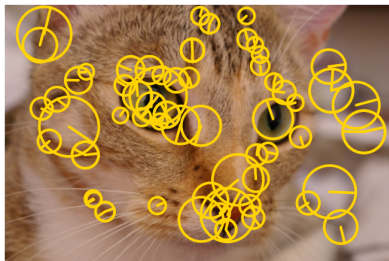
Learn models of simple features: interest points, edgelets

Each interest point is then described by a SIFT descriptor: the region around the point, at the point's scale and rotated to its dominant gradient orientation, is divided into 4×4 cells, and in each cell a histogram of gradient orientations with 8 bins is computed; the 128 numbers are the descriptor. Because the region is taken at the point's own scale and orientation, the descriptor changes little when the object is moved, scaled or rotated.

The figure shows both kinds of feature on a photograph: SIFT key points, each drawn as a circle whose radius is proportional to its scale, with a line showing its orientation; and edgelets from the Canny edge detector (a standard detector that marks thin lines of strong intensity gradient), coloured by orientation: red near-vertical, teal near-horizontal, blue rising to the right, orange falling to the right.

Learn models of simple features: interest points, edgelets

interest points (SIFT)
circle: scale, line: orientation



edgelets
colour: orientation

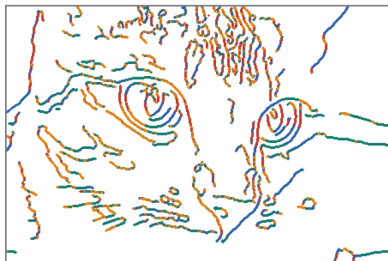


Figure 12: Simple features on the photograph “chelsea” (computed with scikit-image). First panel: the 70 SIFT key points of largest scale; circle radius: 2.5 times the scale, line: orientation. Second panel: edgelets from the Canny edge detector, coloured by four ranges of orientation.

Unsupervised structure induction: everything is unknown

The challenge of unsupervised structure induction is that almost everything is unknown.

- ▶ The graph structure is unknown.
- ▶ Whether an object is present in an image is unknown.
- ▶ How many types of objects can be present is unknown.
- ▶ Which IPs are on the object and which are background is unknown.
- ▶ The correspondence between the IPs of an image and the nodes of the graphical model is unknown.

The object has a cluttered, noisy background. The CMUGL slides compare this with the cocktail party effect: the ability to focus one's listening attention on a single talker among a mixture of conversations and background noises, ignoring the other conversations. Here the single talker is the interest points of the object, and the other conversations are the background.

Unsupervised structure induction: everything is unknown

The correspondence problem is the heart of the difficulty. Some interest points are background; others are from the object, but from which part of the object? If the correspondence were known (this IP is the left eye, that one the nose), learning the model would be easy: average the positions and the descriptors of each part. If the model were known, finding the correspondence would be inference. Both are unknown, so they have to be estimated together, as in the EM algorithm of Lecture 8 (expectation-maximisation), with the correspondence as the hidden variable.

The number of possible correspondences is huge: with n model nodes and m interest points in an image, each node can take any of the m points or none, about $(m + 1)^n$ assignments per image.

Unsupervised structure induction: everything is unknown

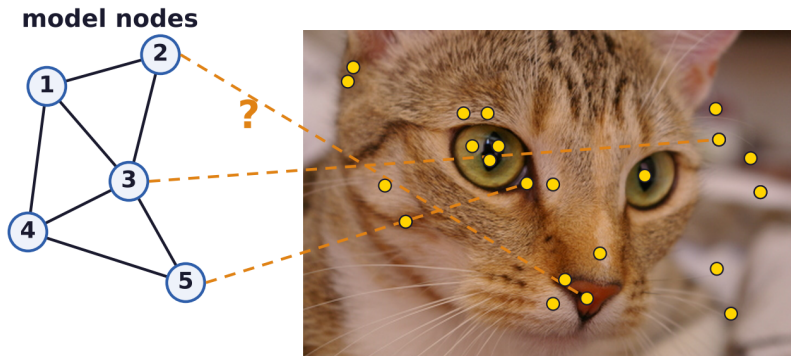


Figure 13: The correspondence problem. A model with five nodes (left) and the interest points of an image (yellow dots, on the photograph “chelsea”). Which point corresponds to which node is unknown; the dashed orange lines are candidate correspondences.

The basic elements are triplets of interest points

The probabilistic grammar-Markov model (PGMM; Zhu, Chen and Yuille, 2009) is built by a graph grammar, a set of rules that add triplets of IPs to a graph. Each point i has a position z_i , a scale s_i and an orientation θ_i , collected in $r_i = \{z_i, s_i, \theta_i\}$ in the CMUGL slides, and an appearance, its SIFT descriptor. Three related points form a triplet $T = \{r_a, r_b, r_c\}$.

A triplet is the smallest group of points that has a shape: the triangle that the three points form, with the orientations of the points relative to its sides. The paper describes this shape by a 15-dimensional vector that is invariant to the scale and rotation of the triplet: the three side lengths divided by the perimeter (3 numbers), the cosine and sine of each angle of the triangle (6), and the cosine and sine of the angle between each point's orientation and a side (6). The scales of the points are left out of it. The same triplet on a larger or rotated object gives the same vector, so the model can recognise it at any scale and orientation.

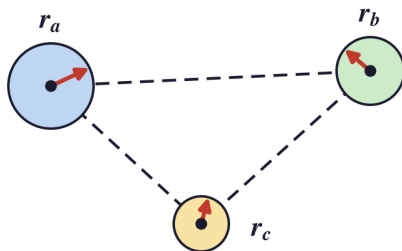
The basic elements are triplets of interest points

The model grows by two rules of the grammar.

- ▶ AND extension: a new triplet is built on two nodes of an existing triplet, plus one new node. The two triplets share an edge, and the model becomes a larger structure of triangles that share edges.
- ▶ OR extension: a new triplet is added as another child of the category node, the root of the model. It starts a new aspect: a separate sub-model for one viewpoint or subclass, such as an aeroplane seen from the side and from the front.

The name says what the model is. The grammar (the AND and OR rules) builds the graph; on the graph, a Markov random field relates the triplets that share nodes, with Gaussian distributions for the invariant triplet vectors and for the descriptors of the nodes.

The basic elements are triplets of interest points



z: position

s: scale (radius)

θ : orientation (red)

Figure 14: A triplet of interest points r_a , r_b , r_c (after CMUGL p. 26, redrawn). Each point has a position z (the black dot), a scale s (the radius of its circle) and an orientation θ (the red arrow).

PGMMs search greedily over model structures

The basic idea of PGMMs is to search over model structures to find the optimal structure. The whole procedure is a greedy search.

1. Initially, all the data are assumed to be generated by a background model, without any spatial relationship between them.
2. Expand the structure by the AND/OR graph grammar. Each member of a triplet vocabulary (about a thousand k -means clusters of triplets from the data), attached by an AND or an OR extension, is a candidate; EM fits its parameters.
3. For each extension, compute a score: the probability that the extended model generates the training images. Accept the extension with the highest score and update the structure.
4. Repeat steps 2 and 3 until the score almost does not change, and exit with the graph structure.

The search is greedy: it never undoes an extension. It may end in a good structure short of the best, but at each step it scores only the extensions of one structure. Points that a new triplet explains move from the background model to the object model.

PGMMs search greedily over model structures

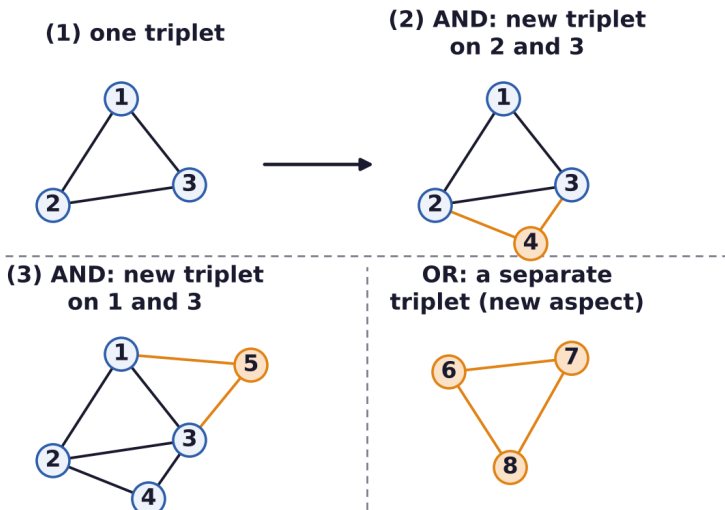


Figure 15: Growing a PGMM (after CMUGL p. 27 and Zhu et al., 2009, redrawn). (1) One triplet. (2) and (3) AND extensions: a new triplet on two nodes of an old one (new node and edges in orange). A separate triplet under the category node is an OR extension: a new aspect.

Model selection scores how well a model generates the data

Model selection is performed by evaluating the probability that the model generates the data. Write $\mathcal{G}$ for a graph structure, Θ for all the parameters of the model with that structure, $D = \{D^i\}$ for the interest points of all the training images, and C for the unknown correspondence of the points of one image to the nodes of the model. Then

$$P(D | \mathcal{G}) = \int d\Theta P(\Theta | \mathcal{G}) \prod_i \sum_C P(D^i | C, \Theta, \mathcal{G}) P(C | \Theta, \mathcal{G}).$$

The sum over C removes the unknown correspondence of each image; the product runs over the training images, which are independent given the parameters; and the integral averages over the parameters with their prior. A structure with a high $P(D | \mathcal{G})$ generates the training data well. Write $P(D | \Theta, \mathcal{G})$ for the product of the sums, the likelihood of the parameters.

Model selection scores how well a model generates the data

The integral over Θ is a big challenge. The PGMM replaces it by the value of the integrand at the parameters Θ^* that maximise the integrand $P(\Theta | \mathcal{G}) P(D | \Theta, \mathcal{G})$:

$$P(D | \mathcal{G}) \approx P(\Theta^* | \mathcal{G}) P(D | \Theta^*, \mathcal{G}),$$

and the structure with the highest score is kept.

The Laplace approximation, the standard approximation of such integrals, keeps one more factor. It approximates the integrand by a Gaussian around Θ^* ; with d parameters and A the matrix of second derivatives of minus the log integrand at Θ^* ,

$$\log P(D | \mathcal{G}) \approx \log [P(\Theta^* | \mathcal{G}) P(D | \Theta^*, \mathcal{G})] + \frac{d}{2} \log 2\pi - \frac{1}{2} \log \det A.$$

The last terms, the volume factor, penalise structures with many parameters that are each well determined by the data: an Occam factor. The PGMM score, the integrand at Θ^* , drops them; the greedy search instead stops when an extension raises the score by too little.

PGMMs learn Caltech 101 categories without part labels

Caltech 101 (Fei-Fei, Fergus and Perona, 2004) is a dataset of images of 101 object categories, each object roughly centred. The PGMM experiments learn one PGMM per class, from images of that class, without part labels and without aligning the images; unsupervised learning on Caltech was unusual.

The task is to classify a test image as containing the object or as background only. The classification rate is taken at the point where the true positive rate equals one minus the false positive rate: 90 % means that 90 % of the object images are detected while 10 % of the background images give false alarms.

The paper first reports 13 categories. To avoid concerns about selection bias, it then uses all 26 classes with at least 80 images, enough data for training and testing. The average classification rate over these 26 classes is 87.6 %.

PGMMs learn Caltech 101 categories without part labels

The PGMM could learn one, two, three or more aspects (children of the OR node at the root) if the dataset required it: for example aeroplanes, faces and bikes. It could learn object models when the training images were a random mix of images with and without the object, because the background model explains the images without it.

The CMUGL slides summarise its performance as comparable to the supervised methods of the time (2006). The paper itself finds its results slightly below those of bag-of-keypoints methods, which need weak supervision: each training image labelled as containing the object or not.

PGMMs learn Caltech 101 categories without part labels

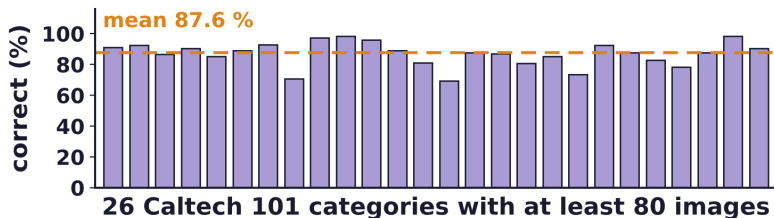


Figure 16: Classification rate of the PGMM for the 26 Caltech 101 categories with at least 80 images (Zhu et al., 2009; bars traced from CMUGL p. 29 and redrawn). The dashed orange line is the paper's mean, 87.6%; the traced bars average 87.0%.

Interest-point models can teach models with more cues

PGMMs use only the features at interest points; the edges between them, the regions they enclose and the appearance of the surface are ignored. The remedy (Chen et al., 2009) is to use the PGMM to learn a skeleton structure of the object, and then use the skeleton to train a model that uses more cues, edges and appearance. The probabilistic object models (POMs) form a chain:

- ▶ POM-IP, the interest-point model (a PGMM), is learnt first;
- ▶ it trains POM-mask, a model of the region of the object, learnt with a segmentation by max-flow / min-cut (a graph-cut method that splits the pixels into object and background at minimum cost);
- ▶ POM-IP and POM-mask together train the POM-edgelets: the mask gives the context in which the edgelet models of the boundary are learnt.

The paper calls this knowledge propagation. On the 26 Caltech classes it reports 86.2 % for POM-IP alone (a PGMM with a pose variable, in an experiment separate from the 87.6 %) and 88.6 % for the full POM: the simple IP model can train a model with more cues.

Interest-point models can teach models with more cues

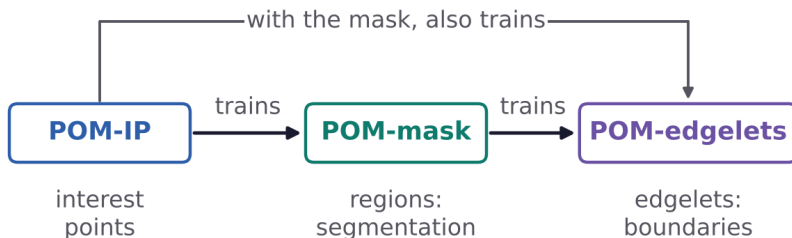


Figure 17: Knowledge propagation between probabilistic object models (after Chen et al., 2009, Fig. 1, redrawn). POM-IP, on interest points, trains POM-mask, on regions; with the mask, it also trains the POM-edgelets, on boundaries.

A hierarchy is learnt from a few unlabelled images

The PGMM grows one graph of triplets, all at one level. The second approach (Zhu, Lin, Huang, Chen and Yuille, ECCV 2008; cited as Zhu et al., ECCV 2008) learns a hierarchy of RCMs with several levels, from short pieces of edge up to the outline of the whole object, one level at a time.

The task is set as hard as possible.

- ▶ Given: a few training images, 12 photographs of horses from the Weizmann horse database (Borenstein and Ullman, 2002), with no labelling and no alignment: the horses are at different positions and sizes, against different backgrounds.
- ▶ The features are highly ambiguous: the edges of the image, of which many belong to the fence, the grass and the stable.
- ▶ To estimate: the graph structure (the nodes and the edges of the hierarchy) and the parameters of its potentials.

The remaining 316 images of the database, with their ground truth segmentations, are kept for testing.

A hierarchy is learnt from a few unlabelled images

The correspondence is unknown, as for the PGMM, and now at every level: which edges of an image form a subpart, which subparts form a part, and which parts form the horse. Trying all matches is a combinatorial explosion. An image has thousands of edge points; even the groups of three nearby pieces of edge number in the hundreds of thousands over the 12 images, and every level multiplies the possibilities.

The way out is to avoid matching whole models to images. Each level is learnt from the level below by a few cheap, local operations: compose nearby pieces, cluster the compositions, keep the clusters that recur in almost every image, and remove those that compete for the same edges. Each operation discards most of the candidates, so the search stays tractable. The model is then refined from the top down, where the bottom-up steps left gaps.

A hierarchy is learnt from a few unlabelled images

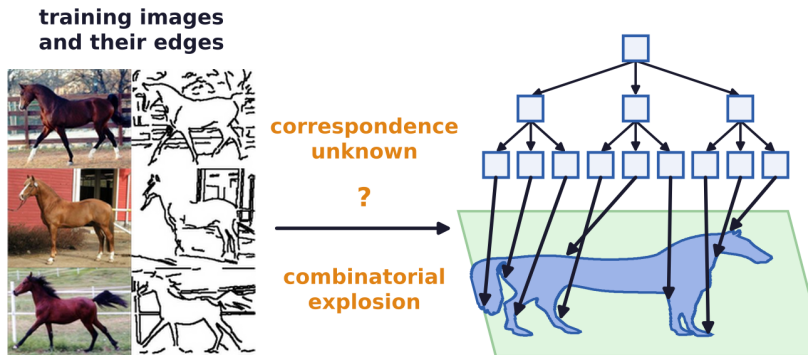


Figure 18: Unsupervised learning of a hierarchy (after CMUGL p. 32, redrawn). First column: three of the 12 training images (Weizmann horse database); second column: their edge maps. The tree on the horse outline stands for the model to be learnt; which edges belong to which node is unknown.

The dictionary: from generic parts to object structures

The learnt hierarchy is a dictionary: a set of concepts at each level. A concept is a dictionary entry, the model of one part. An instance is an occurrence of a concept in an image, at a particular position. A concept at level l is a cluster of compositions of three concepts of level $l - 1$: from each triplet of instances of level $l - 1$, an instance of level l is built whose children are the three.

The levels are numbered as in the table of the paper, L0 to L4.

- ▶ L0: edge points grouped into four orientations, 45 degrees apart (horizontal, vertical and the two diagonals); 4 concepts.
- ▶ L1 and L2: short straight and bent pieces of edge, and corners; 48 and 116 concepts. These are generic: any object with a boundary has them.
- ▶ L3: longer pieces of the outline; 53 concepts.
- ▶ L4: large pieces of the outline (the back and the legs); 2 concepts.

The dictionary bridges the gap between generic features and specific object structures, with one representation (RCMs) and one learning procedure at every level.

The dictionary: from generic parts to object structures

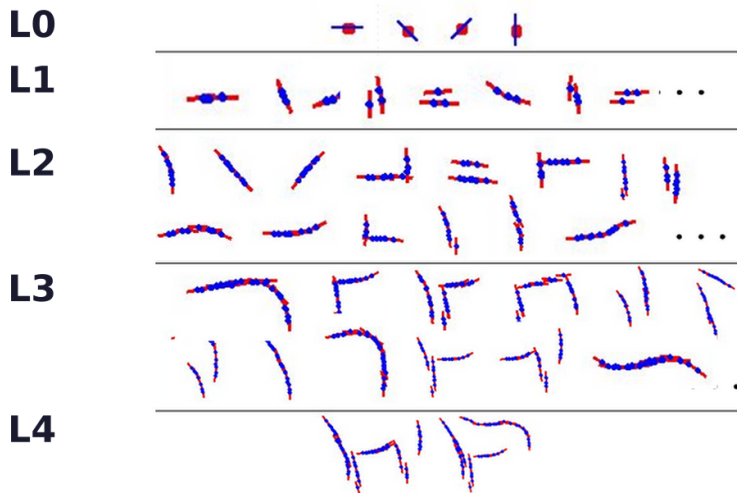


Figure 19: The dictionary learnt from the 12 horse images (Zhu et al., ECCV 2008, Fig. 6, from CMUGL p. 33; level labels retyped). Each entry is the mean shape of a concept: edge points with their orientations. The dots stand for more concepts.

Bottom-up learning composes, clusters, prunes and excludes

Each level is learnt from the instances of the level below by four steps.

1. Composition proposes candidates: every triplet of instances of level $I - 1$ that lie close together is a proposal for an instance of level I .
2. Clustering turns the proposals into concepts. Each triplet is described by its triplet vector, which, as for the PGMM, encodes the shape of the triangle and the orientations of the three pieces. The paper clusters the triplets by the second-order moments (means and covariances) of this vector, so a cluster collects triplets of similar shape.
3. Suspicious coincidence: a concept is kept only if it occurs in at least 90 % of the training images.
4. Competitive exclusion: concepts whose instances seek to explain overlapping parts of the image compete, and the best is kept.

Between clustering and pruning, the paper parses the training images with the candidate concepts, to find all their instances; the counts that the last two steps use come from these parses.

Bottom-up learning composes, clusters, prunes and excludes

Suspicious coincidence is the key test. Three pieces of edge can fall into a particular arrangement by accident, in the clutter of one image. If the same arrangement appears in almost every image, against different backgrounds, the coincidence is suspicious: the arrangement is more likely a property of the object, a part of every horse.

Competitive exclusion removes redundancy. Two concepts can describe nearly the same edges, for example a corner and a slightly larger corner that contains it. Keeping both would double the dictionary without explaining more of the image, so they compete, and only the concept that explains the edges better survives. The name comes from ecology, where two species that compete for the same resources cannot coexist.

The surviving concepts and their instances become the input of the next level, and the four steps are repeated until the top level, where a few concepts cover large parts of the horse. In the schematic, the bars of panel 3 give the fraction of training images in which each concept occurs; in panel 4 two concepts explain the same corner, and the dashed teal one is removed.

Bottom-up learning composes, clusters, prunes and excludes

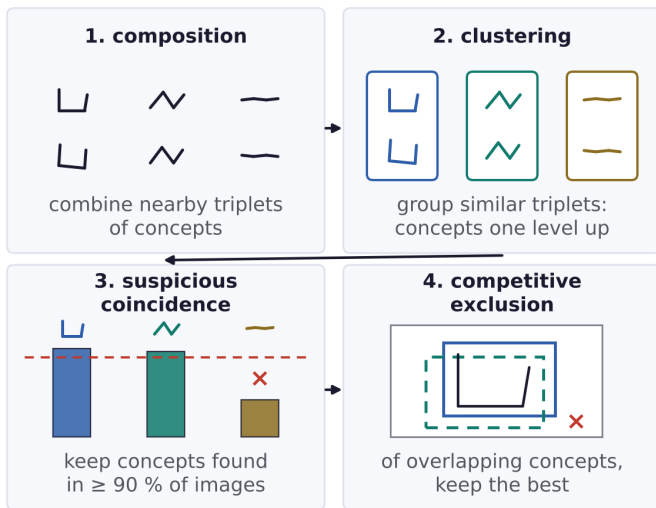


Figure 20: The four steps of bottom-up learning (after CMUGL p. 34; illustrative shapes). 3: dashed red line, the 90 % threshold; red cross, a removed cluster. 4: the solid blue concept is kept.

Dictionary size, part sharing and computational complexity

The table counts, at each level, how many candidates survive each step, over the 12 training images. At level 2, for example, composition proposes 2,034,851 triplets; clustering groups them into 741,662 clusters; suspicious coincidence keeps 995 of them, and competitive exclusion 116. From two million proposals to 116 concepts is a reduction by a factor of about 17,500.

Suspicious coincidence makes the largest cut, as a ratio, at every level: from 14,684 to 262 clusters at level 1 (a factor of 56), from 741,662 to 995 at level 2 (745), from 1,012,777 to 305 at level 3 (3,321) and from 72,620 to 30 at level 4 (2,421). Most compositions are accidents of one image.

The last column gives the learning time of each level in seconds: 480 seconds, or 8 minutes, for the whole hierarchy. Level 2 takes the longest, 254 seconds. Level 3 starts from more concepts (116 against 48) and slightly more compositions, yet takes 99 seconds, and level 4 takes 9. The time is therefore not proportional to the number of compositions; the paper gives the times without a breakdown.

Dictionary size, part sharing and computational complexity

level	composition	clusters	suspicious coincidence	competitive exclusion	seconds
0				4	1
1	167,431	14,684	262	48	117
2	2,034,851	741,662	995	116	254
3	2,135,467	1,012,777	305	53	99
4	236,955	72,620	30	2	9

Figure 21: The number of candidates after each step of learning, at each level (Zhu et al., ECCV 2008, Table 2; from CMUGL p. 35), and the time in seconds. Level 0 groups the edge points into 4 orientation ranges.

Low-level concepts have many instances per image

The chart describes part sharing within the horse. For each level, a point of the curve gives the number of concepts that have a certain number of instances per image, on average over the training images.

The concepts of level 1 have between about 10 and 55 instances in each image: a short straight piece of edge occurs all along the back, the belly and the legs of a horse, so one concept is used many times. Level 2 concepts have fewer, mostly between 5 and 40; level 3 concepts mostly fewer than 10; and level 4 concepts only a few.

So low-level concepts are shared more: many parts of the object reuse them. This is the source of efficiency in a hierarchy. A low-level concept is stored once, and its instances are found once, for all the higher-level parts that use it.

Low-level concepts have many instances per image

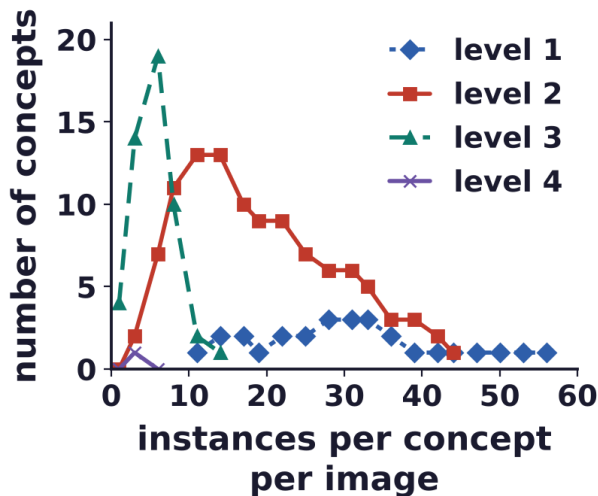


Figure 22: The histogram of concepts at each level (Zhu et al., ECCV 2008, Fig. 6; redrawn from points traced off CMUGL p. 35). A point gives the number of concepts with a certain number of instances per concept per image.

Top-down refinement fills in missing parts

Bottom-up learning leaves gaps in the hierarchy, at the legs, the head and the back, for two reasons. Competitive exclusion can be too strict and remove a part that was needed. And where the legs of a horse join the torso, the local shape varies a lot from horse to horse, so clustering finds no small piece that recurs in 90 % of the images.

Top-down refinement fills in the missing parts. It examines every node of the hierarchy, from the top to the bottom. For each node, a greedy step adds, as a new child, a substructure from the dictionary (the set of proposals obtained in the bottom-up processing), chosen so that the extended structure fits the data best locally; the step is repeated until the gain falls below a threshold. The higher levels act as context, which allows the pruning criteria to be relaxed.

The top-down process also adds a dense representation at the lowest level: many closely spaced edge points along the boundary, so that the model can segment the horse from the background.

Top-down refinement fills in missing parts

The figure shows the result. At the top left is the hierarchy found bottom-up: pieces of the outline with gaps. The arrow leads to the model after top-down processing, whose top node is drawn with coloured rectangles around its parts; the result is the complete outline.

The lower rows show the hierarchy itself, with the parts of each node drawn on a horse. The coloured rectangles highlight the identities of the structures: they mark, on the horse, the parts that each node is made of. The parts in the dashed boxes (leg, head, back) were learnt top-down; all the others were found bottom-up. The dashed arrows lead to them.

Top-down refinement fills in missing parts

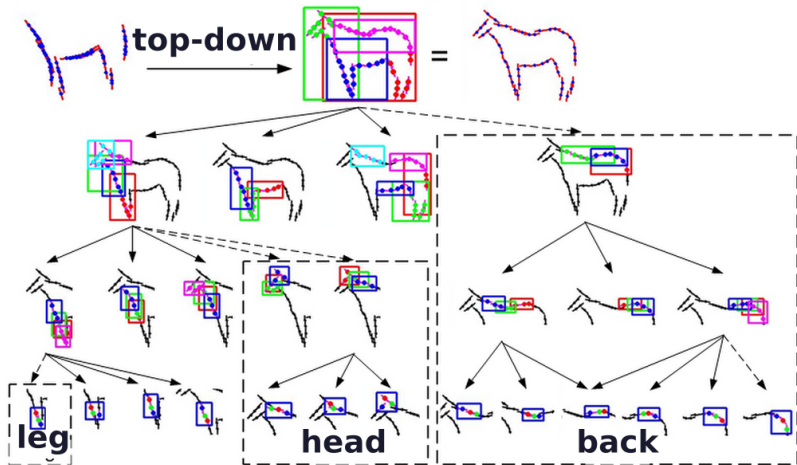


Figure 23: The hierarchy for horses after top-down refinement (Zhu et al., ECCV 2008, Fig. 5; from CMUGL p. 36, labels retyped). Top left: the bottom-up result. Dashed boxes: parts learnt top-down. Coloured rectangles: the parts that make up each node.

Different objects and viewpoints can share parts

A horse hierarchy is one model, for one class in one viewpoint. A visual system must recognise many object classes, each in many viewpoints. The strategy of Zhu et al. (2010) is to share parts between different objects and viewpoints, as in the AND/OR graph of objects A and B, where part b is stored once.

The figure shows why sharing is possible. A car seen from the side, the front and the back gives three very different outlines, and a motorbike gives others; each of these object/viewpoint combinations needs its own model, or template. Yet at the bottom of the hierarchies the pieces are the same: short edges, corners and arcs. Many templates can share these pieces, and some larger ones, such as the round outline of a wheel.

The paper's title, "part and appearance sharing", names a second kind of sharing: the models of one object in different viewpoints share similar appearance cues. Both kinds give a compact set of models, which can be learnt from a limited number of training images.

Different objects and viewpoints can share parts

Learning combines unsupervised and supervised steps.

- ▶ The input is a set of bounding boxes of the different objects/viewpoints and the boundaries of the objects, specified by polygons and line segments.
- ▶ The shared parts are learnt from the object boundaries without object labels: structure induction learns the graph structures and the parameters of the parts that recur in the boundaries.
- ▶ The object models are then built from the shared parts with the labels, and they are supplemented by supervised learning of masks, the regions inside the boundaries, by averaging the boundaries of the training examples of each object and viewpoint.

The cost of a set of models with shared parts is set by the size of the dictionary of parts. Zhu et al. (2010) report that, as templates are added, the number of low-level parts per template falls rapidly: each new template adds mostly the parts it does not share. With separate models the cost would grow in proportion to the number of templates.

Different objects and viewpoints can share parts



Figure 24: Cars and motorbikes in different viewpoints (from CMUGL p. 37): side, front, back and oblique views. Each class looks very different from different viewpoints, and the two classes have similar pieces of outline.

120 templates cover 26 classes in several viewpoints

The test of part sharing is a dataset built from LabelMe (Russell et al., 2008), a database of photographs in which users drew polygons around the objects and named them. Zhu et al. (2010) took 26 object classes from it, including screen, keyboard, mouse, speaker, bottle, mug, chair, can, sofa and lamp, each from several viewpoints. The number of viewpoints varies from class to class. In total there are 120 object/viewpoint combinations, and so 120 templates. The dataset has 843 training images and 896 test images.

The figure shows one silhouette for each of the 120 templates, in 8 rows of 15. Some neighbouring silhouettes are the same class in different viewpoints (cars, chairs, sofas, aeroplanes); others are different classes with similar outlines (a screen and a keyboard are both near-rectangles), which is what makes part sharing worthwhile.

120 templates cover 26 classes in several viewpoints

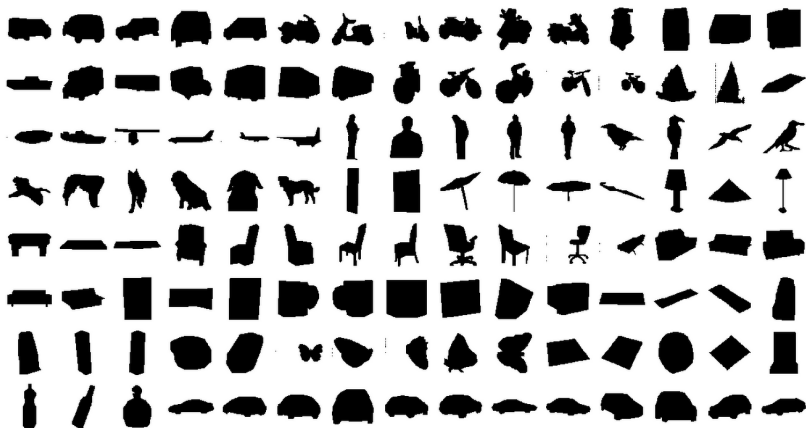


Figure 25: Silhouettes of the 120 object/viewpoint templates (Zhu et al., 2010; from CMUGL p. 39), one per template.

Learn a hierarchical dictionary by suspicious coincidences

The shared parts form a hierarchical dictionary, from low-level through mid-level to high-level parts, learnt from the boundaries of all 120 templates together, without object labels. The procedure follows the horse case, level by level: the orientations of short pieces of boundary are quantised into six bins; triplets of instances that lie close together are composed and clustered; the potentials of the new parts are estimated; overlapping instances are pruned. Learning stops when no new compositions are found. The CMUGL slides call this learning by suspicious coincidences: a kept composition recurs in the training boundaries. A low-level one, such as a corner, recurs in the boundaries of many objects; a high-level one only in those of one object in one viewpoint.

Learn a hierarchical dictionary by suspicious coincidences

The result is a 5-level dictionary, L0 to L4, for the 120 templates. The figure shows the mean shape of each part.

- ▶ L0: pieces of boundary in six orientations, one per bin.
- ▶ L1 and L2: short pieces of edge, corners and bends, generic to all objects.
- ▶ L3: longer pieces of outline, such as the top of a screen or the back of a chair.
- ▶ L4: whole outlines, many of them recognisable: a chair, a lamp, a bottle, a bird, an umbrella.

Learn a hierarchical dictionary by suspicious coincidences

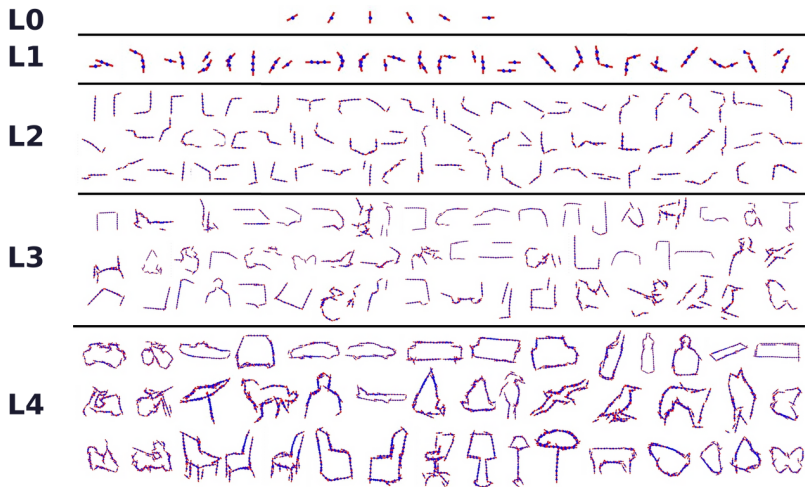


Figure 26: The 5-level dictionary learnt from the 120 templates of 26 classes (Zhu et al., 2010, Fig. 3; from CMUGL p. 40, level labels added). Each level is a band between the black lines, of up to three rows; each entry is the mean shape of a part.

Part sharing decreases with levels

The part-sharing matrix shows which templates use which parts. Its columns are the 120 object/viewpoint models; its rows are some of the parts of the dictionary, grouped by level, with level 1 at the top. A black square means that the part (row) is used by the model (column).

The level 1 block is almost entirely black: the parts of the lowest level are used by almost all the objects and viewpoints. The level 2 block is dense in its first rows and sparser below, level 3 is sparse, and level 4 is almost white: a high-level part is used by one or a few models, because it is close to the outline of one object in one viewpoint.

So part sharing decreases with levels: low-level parts are generic and shared, high-level parts are specific. The same trend appeared within the single horse model, where low-level concepts had the most instances per image. For many objects it means that the low levels of the dictionary are paid for once, and each new template adds mostly high-level parts.

Part sharing decreases with levels

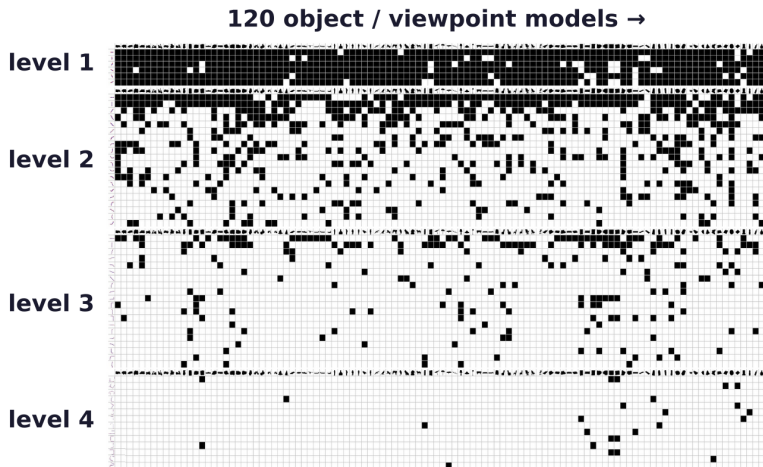


Figure 27: The part-sharing matrix (Zhu et al., 2010, Fig. 9; from CMUGL p. 41, level labels added). Columns: the 120 object/viewpoint models; rows: some of the parts of levels 1 to 4. Black: the part is used by the model. The tiny shapes along the band edges are the original's thumbnails.

RCMs are comparable to the state of the art on single classes

Detection is measured by curves that trade correct detections against false alarms, as the receiver operating characteristic (ROC) curves of Lecture 3.

- ▶ Precision: the fraction of the detections that are correct. Recall: the fraction of the objects that are found. Lowering the detection threshold raises the recall and usually lowers the precision, which traces a precision-recall curve.
- ▶ AP (average precision): the area under the precision-recall curve, the average of the precision over recall, given in percent; higher is better.
- ▶ False positives per image: the number of wrong detections per image. A curve of recall against false positives per image is summarised by its AUC (area under the curve).

The competitors include HOG + SVM (Dalal and Triggs, 2005): histograms of oriented gradients (HOG), histograms of gradient orientations in the cells of a window like those of the SIFT descriptor, classified by a linear support vector machine (SVM).

RCMs are comparable to the state of the art on single classes

The RCMs were also tested on three datasets with a single class each, against the best methods of the time. A separate family of RCMs, one per viewpoint, was trained on each dataset.

- ▶ (a) Motorbikes (PASCAL 2005) in 16 viewpoints, 458 training and 179 test images: the RCMs reach an AP of 73.8, against the methods of Thomas et al. (2006) and of Savarese and Fei-Fei (2007). The RCM curve keeps a higher precision over most of the range of recall.
- ▶ (b) Weizmann horses in 5 viewpoints, 50 training, 50 validation and 228 test images: the RCMs reach an AUC of 98.0, against 93 for Shotton, Blake and Cipolla (2008). The RCMs find almost every horse at very few false positives per image.
- ▶ (c) LabelMe cars in 12 viewpoints, 160 training and 800 test images: the RCMs reach an AP of 66.6, against 63.2 for HOG + SVM.

On these single-class datasets the RCMs match or beat the best methods of the time.

RCMs are comparable to the state of the art on single classes

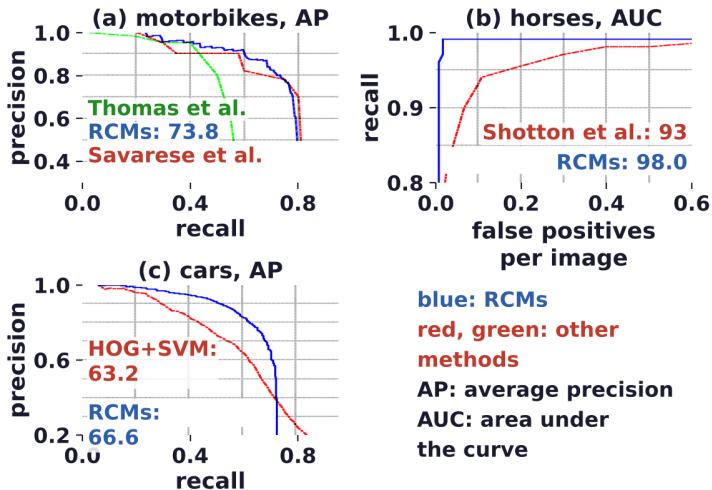


Figure 28: Detection curves on three single-class datasets (Zhu et al., 2010, Fig. 5; from CMUGL p. 42, labels retyped). (a) and (c): precision against recall; (b): recall against false positives per image.

Unsupervised structure learning is possible but very hard

The conclusion of this part of the lecture is that unsupervised structure learning is possible, but very difficult. The PGMMs grew graphs of interest points from images of one class, without part labels; the hierarchy of RCMs learnt a dictionary of parts from 12 unaligned horse images; and a second dictionary, learnt from the annotated object boundaries of 26 LabelMe classes, is shared by 120 templates. Each needed careful design: invariant triplet vectors, thresholds such as the 90 % of suspicious coincidence, and top-down repairs where the bottom-up search failed.

The CMUGL slides ask a question: can limited additional information make structure learning much easier? One annotated example of the object (one image in which the parts are marked) would fix the correspondence for that image, and the other images could be matched to it. That is far less than the full supervision that deep networks use, and it could remove much of the search.

Unsupervised structure learning is possible but very hard

How do humans learn object structure? Humans learn in a life-long manner since infancy. Infants learn by interacting with the world, touching and playing with objects, which is more than seeing many images. When an infant picks up a toy and turns it, the parts move together, the views change smoothly and the hidden sides appear: the correspondence between views, and the grouping of parts into one object, are given by the action.

The static image collections of these experiments lack this information, which is one reason the learning is so hard. Video, motion, depth and interaction are natural sources of the extra information that the question asks for.

Summary

- ▶ Objects are compositions of semantic parts and subparts: a hierarchical graph.
- ▶ A probability model on it: graph, state variables, potentials and weights; an exponential (Gibbs) model.
- ▶ Three tasks: inference (bottom-up and top-down), learning the weights, and structure induction.
- ▶ AND/OR graphs: AND nodes compose parts, OR nodes switch them for different poses and viewpoints.
- ▶ Unsupervised structure learning: PGMMs grow the graph triplet by triplet while the score improves.
- ▶ Bottom-up: compose, cluster, keep suspicious coincidences, exclude competitors; then refine top-down.
- ▶ Sharing parts across objects and viewpoints makes many models efficient; sharing decreases with level.
- ▶ Compositional models are challenging, but multi-task, interpretable and promising for domain transfer.

Summary

The common thread is explicit structure. An object is a graph of parts with a probability model on it; the same graph is used to detect the object, to locate its parts and to trace its boundary, and its parse says which parts were found and where. OR nodes and shared parts let one graph cover many poses, viewpoints and objects, at a cost set by the number of distinct parts.

Learning the graph is the hard problem. The PGMM grows a graph of interest points greedily, scored by how well it generates the data. The hierarchical dictionary is learnt level by level, by composition, clustering, suspicious coincidence and competitive exclusion, and then repaired top-down. Both work without part labels, and both need careful design. The CompNets of the first part of the lecture take a middle path: deep network features learnt with supervision, and an explicit compositional model of parts on top of them.

References

- [1] A. L. Yuille. Compositional (semantic) models and unsupervised graph structure learning. Course slides (CMUGL), Johns Hopkins University.
- [2] M. A. Fischler and R. A. Elschlager. The representation and matching of pictorial structures. *IEEE Transactions on Computers*, C-22(1):67–92, 1973.
- [3] L. Zhu, Y. Chen and A. Yuille. Recursive compositional models for vision: description and review of recent work. *Journal of Mathematical Imaging and Vision*, 41(1–2):122–146, 2011.
- [4] L. Zhu, Y. Chen, A. Torralba, W. Freeman and A. Yuille. Part and appearance sharing: recursive compositional models for multi-view multi-object detection. *CVPR*, 1919–1926, 2010.
- [5] S.-C. Zhu and D. Mumford. A stochastic grammar of images. *Foundations and Trends in Computer Graphics and Vision*, 2(4):259–362, 2007.
- [6] A. L. Yuille and C. Liu. Deep nets: what have they ever done for vision? *International Journal of Computer Vision*, 129(3):781–802, 2021.
- [7] L. Zhu, Y. Chen and A. Yuille. Learning a hierarchical deformable template for rapid deformable object parsing. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(6):1029–1043, 2010.

References

- [8] L. Zhu, Y. Chen, C. Lin and A. Yuille. Max margin learning of hierarchical configural deformable templates (HCDTs) for efficient object parsing and pose estimation. *International Journal of Computer Vision*, 93(1):1–21, 2011.
- [9] L. Zhu, Y. Chen, Y. Lu, C. Lin and A. Yuille. Max margin AND/OR graph learning for parsing the human body. *CVPR*, 1–8, 2008.
- [10] T. Kadir and M. Brady. Saliency, scale and image description. *International Journal of Computer Vision*, 45(2):83–105, 2001.
- [11] D. G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60(2):91–110, 2004.
- [12] L. Zhu, Y. Chen and A. Yuille. Unsupervised learning of probabilistic grammar-Markov models for object categories. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 31(1):114–128, 2009.
- [13] L. Fei-Fei, R. Fergus and P. Perona. Learning generative visual models from few training examples: an incremental Bayesian approach tested on 101 object categories. *CVPR Workshop on Generative-Model Based Vision*, 2004.

References

- [14] Y. Chen, L. Zhu, A. Yuille and H. Zhang. Unsupervised learning of probabilistic object models (POMs) for object classification, segmentation, and recognition using knowledge propagation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 31(10):1747–1761, 2009.
- [15] L. Zhu, C. Lin, H. Huang, Y. Chen and A. Yuille. Unsupervised structure learning: hierarchical recursive composition, suspicious coincidence and competitive exclusion. *ECCV*, LNCS 5303, 759–773, 2008.
- [16] E. Borenstein and S. Ullman. Class-specific, top-down segmentation. *ECCV*, LNCS 2351, 109–124, 2002.
- [17] B. C. Russell, A. Torralba, K. P. Murphy and W. T. Freeman. LabelMe: a database and web-based tool for image annotation. *International Journal of Computer Vision*, 77(1–3):157–173, 2008.

References

- [18] A. Thomas, V. Ferrari, B. Leibe, T. Tuytelaars, B. Schiele and L. Van Gool. Towards multi-view object class detection. *CVPR*, 1589–1596, 2006.
- [19] S. Savarese and L. Fei-Fei. 3D generic object categorization, localization and pose estimation. *ICCV*, 1–8, 2007.
- [20] J. Shotton, A. Blake and R. Cipolla. Efficiently combining contour and texture cues for object recognition. *BMVC*, 2008.
- [21] N. Dalal and B. Triggs. Histograms of oriented gradients for human detection. *CVPR*, 886–893, 2005.